

Classroom Assignment: Understanding JavaScript Scoping (var vs let vs global)

Learning Objective:

Understand **global scope**, **function scope**, and **block scope** by using var and let inside conditional blocks.

Expected Completion Time:

Best Case: ***15 minutes***

Average Case: ***20 minutes***

Assignment Details:

Write a JavaScript program to observe how the same variable name behaves:

- * in ***global scope***,
- * inside a ***function***, and
- * inside an ***if-block*** using both var and let.

Assignment Requirements:

1. Declare a ***global variable*** named genderType with value "female".
2. Create a function named ***printGender***.
3. Inside the function, declare a ***function-scoped*** variable color with value "brown" using let.
4. Create an ***if condition*** that checks whether genderType starts with "female".
5. Inside this if-block:
 - * Declare a variable age = 30 using ***var***.
 - * Create a ***block-scoped*** variable color = "pink" using let.
 - * Print the color inside the block and observe which value appears.
6. Outside the if-block but inside the function, print the value of age.
7. Call the function and print the value of genderType globally.
8. Now change the global variable named **genderType** with value "male" and observe the functionality of scoping in JavaScript.

Hints to Solve:

- * var inside a block acts as if declared at the *function level*.
- * let inside a block is visible *only inside that block*.
- * Try printing color from different places to observe which value is picked.
- * Compare the behavior of the same variable name "color" when declared with let inside function vs inside block.

Expected Outcome:

After completing this assignment, you should be able to

- * Understand how *global variables* can be accessed inside functions.
- * Differentiate *function-scope* (var) from *block-scope* (let).
- * Explain why the color variable prints "pink" inside the block but remains "brown" outside.
- * Understand why age is accessible outside the if-block.